Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 00 (2026) 000–000

Procedia Computer Science

www.elsevier.com/locate/procedia

5th International Conference on Machine Learning and Data Engineering

EchoPath: A Real-Time Assistive Indoor Navigation System For The Visually Impaired Using Computer Vision

Abstract

People with visual impairments face significant challenges when navigating complex indoor environments independently. India has over 35 million visually impaired individuals, many of them of college-going age, and successive All India Survey on Higher Education (AISHE) reports record a steep decline in the number of enrolled students with disabilities between 2019 and 2022. This paper proposes EchoPath, an infrastructure-free assistive navigation system that combines a fine-tuned YOLOv8n detector, trained on a 17-class indoor dataset assembled from COCO 2017 and public Roboflow Universe datasets, with MiDaS Small depth estimation and a priority-based text-to-speech (TTS) alert pipeline. We report the submitted model (Training v3, mAP@50 of 0.869 on its own validation split) and a revised model (Training v4), retrained because the v3 classes were strongly imbalanced, on a rebuilt dataset with importance-weighted class balancing, attributed sources, no training–validation leakage and completed labels. On identical leakage-free validation images, v4 raises the mean mAP@50 over the 17 classes from 0.608 to 0.813, and recall at the deployment threshold for stairs from 0.25 to 0.87, waste bins from 0.08 to 0.95 and elevators from 0.63 to 0.96. On an RTX 4500 Ada GPU the full pipeline processes 13.9 frames per second, with a median of 0.62 s from frame capture to the start of speech. In 13 recorded walkthroughs, 77% of spoken alerts are correct, but 57% of the obstacles in the walking path are not announced, mostly because they are not detected. YOLOv8n runs 3.7 times faster than RT-DETR-L with a tenth of its weights, while RT-DETR-L is more accurate before fine-tuning.

© 2026 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Machine Learning and Data Engineering (ICMLDE2026).

Keywords: Object detection; YOLOv8; Assistive technology; Visual impairment; Audio navigation; Real-time detection; Indoor navigation; Text-to-speech; Monocular depth estimation

1. Introduction

According to the World Health Organization, 2.2 billion people have some level of visual impairment [1]. India has approximately 4.95 million blind people and an estimated 35 million with some degree of visual impairment, including about 0.24 million blind children, and the majority of people with visual impairment are between 18 and 24 years old, the typical age for entering college [2]. Yet of approximately 92,000 students with an identified disability enrolled in Indian colleges [3], only an estimated 25,000 to 35,000 are visually impaired [4, 5]. Mobility and orientation are among the most common challenges of these students, many of whom depend on classmates to move around campus [6], and independent campus navigation was identified as a primary barrier to higher education at Aligarh Muslim University [7]. Object detection has advanced considerably with the YOLO family [8]; YOLOv8n, in the Ultralytics library [9], offers a favourable trade-off between accuracy and computational cost.

1877-0509 © 2026 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Machine Learning and Data Engineering (ICMLDE2026).

EchoPath runs on a GPU workstation, with the NVIDIA Jetson Nano as the planned deployment platform, and delivers guidance through a TTS interface driven by a fine-tuned YOLOv8n detector. Four training runs were performed: v1 (26-class output head, 24 classes with validation labels), v2 and v3 (19-class head, 17 evaluated classes; v3 is the submitted model) and v4, the revised 17-class model, retrained on an audited and rebalanced dataset because the v3 classes were strongly imbalanced. The objectives are (1) four iterative YOLOv8n fine-tuning sessions with a per-class analysis; (2) a comparison of YOLOv8n with RT-DETR [10] across deployment metrics; (3) a data audit (source attribution, removal of training–validation near-duplicates, importance-weighted class balancing) and its measured effect on per-class detection; and (4) an end-to-end architecture integrating detection, depth and speech.

Related work. Early ultrasonic and infrared aids [14] offered little semantic detail indoors. YOLO-based detection reached an average accuracy of 91.7% over nine obstacle types in a navigation prototype for visually impaired users [15], whereas RT-DETR [10] offers high mean average precision (mAP) at a much higher computational cost, and monocular depth models such as MiDaS generalise to unseen datasets without fine-tuning [12]. NavCog's Bluetooth beacons reduce cognitive load but require installation [16], and non-speech spatial sounds can outperform verbal commands [17]. Camera-based systems are accurate but often too demanding for economical devices [18]; VISA [19] combined indoor positioning, object detection and text recognition in a far more complex system. In India, MAVI [20] supports outdoor mobility (RGB camera, GPS, IMU) without indoor directional guidance, Kasture et al. [21] used YOLOv3 for auditory object descriptions without directions, and Sharma et al. [22] built ultrasonic sidewalk devices. Little work targets academic buildings, with notice boards, whiteboards, water dispensers and elevator panels. Unlike Microsoft Seeing AI and Google Lookout, which lack continuous navigation and directional guidance, EchoPath needs no infrastructure (no beacons or prior building configuration), fuses detection with depth estimation, gives cues for three zones (left, ahead, right) and speaks only when necessary to limit cognitive load.

2. Methodology

2.1. System architecture

EchoPath has three stages (Fig. 1). Stage 1 captures video at 30 frames per second; the 13 evaluation videos were recorded with a smartphone at chest height at 1080×1920 pixels (portrait; one at 480×864), which YOLOv8n resizes to 640 pixels internally; deployment will use a chest-mounted camera. Stage 2 runs the fine-tuned YOLOv8n [9] and MiDaS Small [12] on each processed frame, and Stage 3 speaks the class, zone and direction of a detection through earphones, 0.62 s after frame capture in the median and 3.6 s at the 95th percentile (Section 3.3).

Relative depth. EchoPath uses relative depth, not distance in metres: MiDaS predicts relative inverse depth, defined only up to an unknown scale and shift, and no calibration against measured distances is performed. Each depth map is rescaled per frame to $[0, 1]$, 1 being the nearest point of the frame (Fig. 4). The median of the inner 50% of each detected box (clipped to the frame) is compared with the frame's 75th and 50th percentiles: above the 75th the object is *close*, above the 50th *medium*, otherwise *far*. "Close" therefore means nearer than three quarters of the visible scene, so the same object at the same distance can fall into different zones in different scenes. The direction is taken from the box centre, split at one and two thirds of the frame width. No filter restricts alerts to the walking path: any close or medium object on the left, ahead or right can produce an alert, and its side is spoken so that the user can judge whether it is in the way (path-aware alternatives are compared in Section 3.2).

2.2. Audio output module

Alerts are spoken offline with `pyttsx3` and state the object, zone and side (e.g. "chair close on your right"); far objects are only drawn on the output frame. Because a phrase takes about 2.3 s, a producer–consumer design keeps detection running: the main thread queues alerts and a consumer thread speaks them. The consumer creates a new engine per phrase, because a reused SAPI5 engine silently skips every phrase after its first; this adds about 0.1 s. In each processed frame one candidate is considered (a close object if any, otherwise a medium one; the most confident within a zone), and a new alert is raised only when its description (class, zone, direction) changes. The speaker drops an alert within 1.5 s of the previous spoken one and does not repeat a phrase within 5 s (the voiced evaluation videos

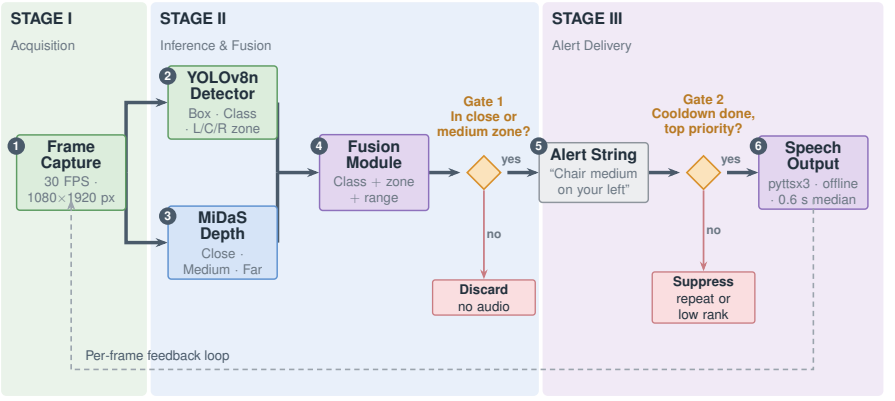


Fig. 1. Architecture of EchoPath: (1) frame capture; (2, 3) YOLOv8n detection and MiDaS Small depth estimation; (4) fusion into class, lateral zone and proximity; (5) alert string; (6) offline speech. Gate 1 discards objects outside the close and medium zones and Gate 2 alerts within the cooldown or outranked by a nearer obstacle; the dashed line is the per-frame loop.

Table 1. YOLOv8n versus RT-DETR-L under the setup of Section 2.3 (median / 95th percentile over 390 frames). Accuracy on the five classes shared with COCO (v4 validation split); the COCO models are not fine-tuned.

Metric	YOLOv8n (COCO)	RT-DETR-L (COCO)	YOLOv8n (EchoPath v4)
Weights (MB) / parameters (millions)	6.5 / 3.2	66.5 / 32.1	6.3 / 3.0
Computation (GFLOPs)	8.7	103.8	8.1
Time per frame (ms), of which network inference	8.1 / 14.2, 5.4	29.6 / 46.4, 26.2	7.6 / 10.7, 5.4
Frames per second	123	34	131
mAP@50 / mAP@50-95, 5 shared classes	0.355 / 0.222	0.410 / 0.272	0.575 / 0.376

use a 2 s gap). These limits follow findings that repeated notifications overload a moving user [17]. The audio layer only receives labels, zones and directions, so it can be replaced (e.g. by haptics) without changing detection or depth.

2.3. *Model selection: YOLOv8n versus RT-DETR*

Four deployment constraints were set: throughput above 25 FPS, model size below 10 MB, latency below 50 ms per frame, and accuracy on safety-critical indoor classes. The original comparison timed the stock models on one video in a loop that also drew and displayed every frame. It was repeated under one setup (Table 1) for stock COCO-pretrained YOLOv8n and RT-DETR-L [10] (the weights used originally) and the fine-tuned EchoPath YOLOv8n: NVIDIA RTX 4500 Ada, Ultralytics 8.4.37, PyTorch 2.4.1+cu118 (CUDA 11.8), 640-pixel input, batch size 1, FP32 (FP16 gave no speed-up), confidence 0.40, the same 390 pre-decoded frames of the 13 videos after 50 warm-up predictions, GPU synchronised around every call, two alternating rounds. Accuracy is the mAP@50 on the v4 validation split for the five classes COCO shares with EchoPath (person, chair, potted plant, table, monitor). YOLOv8n meets all four constraints; RT-DETR-L fails only the size constraint, not three of four as originally stated, and is the more accurate detector before fine-tuning. YOLOv8n was chosen because it needs a tenth of the weights and a twelfth of the computation and runs 3.7 times faster, which matters most on the planned Jetson Nano (not measured here), in line with the priority of efficient methods on low-cost edge devices [18]. Fine-tuning raises YOLOv8n's mAP@50 on the shared classes to 0.575; RT-DETR-L was not fine-tuned, so whether its advantage would persist is not known.

2.4. *Dataset construction and object classes*

Sources. No training images were captured by the authors. All come from COCO 2017 [11] (CC BY 4.0 annotations; each image keeps its Flickr licence) and 18 Roboflow Universe datasets under CC BY 4.0 [24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41], each supplying one or two classes; Supplementary Table S1 lists the

Table 2. Dataset versions and results on each run's own validation split. "Head" is the number of output classes, "eval." the number with validation labels; leakage is the share of validation images with a near-duplicate (Hamming distance ≤ 4) in training. v2's data are reconstructed from an archived copy made before table and monitor were added (21,601 / 2,352 / 1,211 images) plus the table and monitor images identified in v3: validation is exact, training and test sizes are derived from the logged batches per epoch, and leakage is an upper bound. Absolute changes: v1 to v2 +0.068 P, +0.134 R, +0.107 mAP@50, +0.108 mAP@50-95; v2 to v3 −0.014, −0.040, −0.039 (−4.3% relative) and −0.042.

Version	Head / eval.	Train	Val.	Test	Val. labels	Leakage	P / R	mAP@50 / 50-95	Purpose of the run
v1	26 / 24	23,445	2,865	1,367	5,046	42.8%	0.833 / 0.727	0.801 / 0.633	Baseline
v2	19 / 17	≈23,700	2,580	≈1,340	4,453	≤26.6%	0.901 / 0.861	0.908 / 0.741	Fewer classes, new sources
v3 (submitted)	19 / 17	25,666	2,780	–	5,017	25.6%	0.887 / 0.821	0.869 / 0.699	Adds 800 COCO chair images
v4 (revised)	17 / 17	20,307	2,922	–	5,863	0% (removed)	0.855 / 0.787	0.828 / 0.664	Rebalanced, audited data

author, URL, version, licence, classes, image counts and our changes (remapped names, dropped classes, merges) of every source. The 17 classes are listed by tier in Section 2.5. The 13 evaluation videos, recorded by the authors in the Academic Block Building, contain no training images.

Splits (Table 2). Every version kept each source's predefined split (COCO train2017 to training, val2017 to validation); nothing was re-split, and frames of one source video were not grouped. Backpack (1,238 v3 training labels) and handbag (6) had no validation labels in v1 to v3, hence the evaluated classes in Table 2. In v1 and v2 the sources' test images formed a test split; from v3 on they were merged into training (v3: 36,765 training labels). A test split was not needed because (1) no hyperparameter was tuned on the validation split (Section 2.5); (2) the validation split only selects the best epoch, and the best and final epochs differ by at most 0.0014 in mAP@50 in any run (they coincide for v1 and v3), so validation results are effectively held-out results; and (3) a test split from the same public sources would measure the same distribution, whereas the relevant test is footage from the target building, which no source contains: the 13 videos, whose annotation as a labelled test set is planned (Section 4).

Audit of v3. (i) Leakage: 25.6% of v3 validation images have a near-duplicate in training (difference hash, Hamming distance ≤ 4; 8.4% identical), mostly consecutive video frames that the sources placed in both splits, concentrated in person (87% of validation labels), water_dispenser (93%), slide (56%), whiteboard (51%) and notice_board (47%); the archived v1 and v2 datasets give 42.8% and at most 26.6%, so leakage inflated the validation results of v1 to v3. (ii) The sources of the stairs, elevator, potted_plant and monitor data could not be identified. (iii) The 769 COCO chair images carried only chair labels, although COCO annotates 2,364 people, 463 dining tables, 197 potted plants and 108 televisions in them, which the model learned as background. (iv) The class counts were mislabelled in the original submission: the largest v3 class is fire_extinguisher, not column (Fig. 2a).

Why v4 was retrained. The v3 classes were strongly imbalanced (10,929 fire_extinguisher against 106 waste_bin labels, 103:1), reflecting the sizes of the merged datasets, inflated by pre-augmented copies in some sources, rather than each class's importance for navigation. The v4 training split is therefore balanced by importance (Section 2.5), and the rebuild also fixed the other findings: (1) the four unattributable sources and the backpack/handbag source were removed (17 classes); (2) real-world data were added for stairs and elevator [27, 30], monitor [41], potted plant [34] and waste bin [36, 37], plus 311 indoor COCO potted-plant images licensed CC BY 2.0, "no known copyright restrictions" or United States Government work; six visually inspected candidates were rejected (floor plans, toy miniature scenes, catalogue and watermarked stock photographs, printed elevator pictures, strong fisheye distortion); (3) COCO's person, chair, potted plant, dining table and television labels were added to every COCO image; and (4) all 2,727 training images within Hamming distance 4 of a validation image were removed, leaving the validation split unchanged (v4: 33,539 training and 5,863 validation labels).

2.5. Training configuration and hyperparameter tuning

All runs fine-tuned COCO-pretrained YOLOv8n [9] for 100 epochs at 640 × 640 with batch size 64 (Ultralytics 8.4.37, PyTorch 2.4.1+cu118, NVIDIA RTX 4500 Ada, 24 GB). The automatic optimiser selected MuSGD (momentum 0.9, weight decay 0.0005); after a three-epoch warm-up the learning rate of the first parameter group peaked

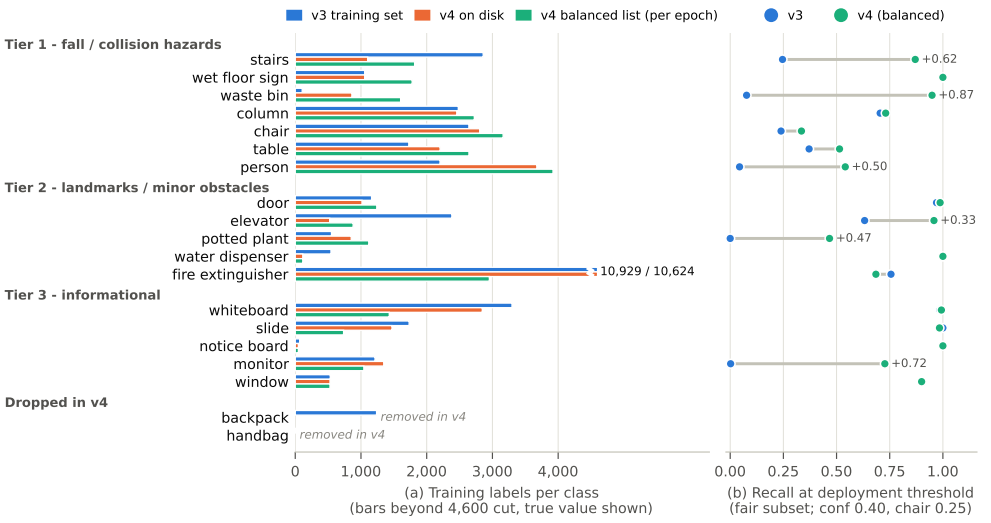


Fig. 2. (a) Training labels per class for v3 and v4, by importance tier (Section 2.5); the v3 imbalance is why v4 was retrained, and the v4 balanced list is what v4 sees per epoch. (b) Recall at the deployment threshold (confidence 0.40; chair 0.25) of v3 and v4 on the fair validation subset (no near-duplicate in either training split); changes ≥ 0.3 are printed.

at 0.03 (three times the base rate of 0.01 of the other groups) and decayed linearly to 0.0006. Augmentation was identical in every run: mosaic ($p = 1.0$, off for the last 10 epochs), horizontal flips ($p = 0.5$), HSV jitter (0.015, 0.7, 0.4), translation 0.1 and scaling 0.5; no mixup or copy-paste was used in any run, contrary to the original statement for v3. v2 and v3 therefore differ only in their data (v3 adds 800 COCO chair images, 600 for training and 200 for validation, and merges v2's test images into training); the v1-to-v2 gain coincides with the reduction from 26 to 19 classes and the revised sources and, as the validation splits differ, cannot be attributed to a single factor.

Importance-weighted class balancing (v4 training split only). Tier 1 holds fall and collision hazards (stairs, wet_floor_sign, waste_bin, column, chair, table, person), tier 2 landmarks and minor obstacles (door, elevator, potted_plant, water_dispenser, fire_extinguisher) and tier 3 informational classes, announced like the others but least important for safe walking (whiteboard, slide, notice_board, monitor, window). Images containing only over-represented, lower-importance classes were first undersampled with a fixed seed (fire_extinguisher to about 3,000 labels, monitor to about 1,000, half of the whiteboard- or slide-only images). Repeat-factor sampling then gives each class $r_c = \min\left(3, \sqrt{T_c/f_c}\right)$, with label count f_c and T_c of 3,000 for tier 1 and 1,500 for tier 2 (tier 3 and the synthetic water_dispenser source are never repeated), and repeats each image by the largest r_c of its labels with stochastic rounding. The 17,110-entry list draws on 14,022 unique images, so no image is duplicated on disk, and random mosaic, flip and colour augmentation make each repeat look different (Fig. 2a, Table 3).

3. Experimental Results

3.1. Detection results

v2 and v3. Their own-split results (Table 2) are not comparable, because v3's split adds 200 COCO chair images to v2's. On v3's split v2 scores 0.848 and v3 0.869 (0.813 and 0.837 without validation images that near-duplicate a training image), so v3 was the stronger model and was retained; v2's originally reported chair recall of 1.000 came from a split without COCO chair scenes (0.152 versus 0.239 for v3 on the shared split). Validation loss was still decreasing at epoch 100 in v1 to v3, so no run overfitted. For v3, the PR curve gives an mAP@50 of 0.995 for person, whiteboard, notice_board and water_dispenser, the four classes with the highest leakage, so these scores overstate real-world performance; precision is 1.00 at maximum confidence, recall 0.94 at confidence 0, and F1 peaks at 0.84 at confidence 0.448.

Table 3. Per-class results of Training v3 (submitted) and Training v4 (revised, class-balanced), each on its own validation split; P and R at the confidence that maximises each run's mean F1. For v4 the table also gives the training labels on disk and per epoch in the balanced list, the validation labels, and P and R at the deployment threshold of 0.40; the v3 and v4 splits differ (Section 3.1).

	Training v3 (17 cls) - Submitted				Training v4 (17 cls) - Revised								
					Labels							At 0.40	
Class	P	R	mAP50	50-95	Train	Bal.	Val.	P	R	mAP50	50-95	P	R
all	0.887	0.821	0.869	0.699	33,539	27,751	5,863	0.855	0.787	0.828	0.664		
person	0.974	0.990	0.995	0.815	3,676	3,922	743	0.769	0.598	0.694	0.433	0.697	0.645
chair	0.746	0.241	0.354	0.260	2,807	3,163	666	0.680	0.287	0.388	0.284	0.595	0.311
door	0.959	0.972	0.989	0.891	1,016	1,241	292	0.957	0.979	0.989	0.892	0.947	0.990
stairs	0.870	0.792	0.897	0.723	1,102	1,817	138	0.921	0.848	0.912	0.597	0.898	0.870
slide	0.989	1.000	0.995	0.983	1,472	736	147	0.970	0.993	0.994	0.963	0.969	0.993
whiteboard	0.993	0.993	0.995	0.980	2,846	1,431	301	0.996	0.990	0.995	0.963	0.994	0.990
notice_board	0.945	1.000	0.995	0.995	48	48	17	0.894	1.000	0.995	0.964	0.881	1.000
elevator	0.908	0.917	0.955	0.653	524	881	95	0.980	0.937	0.965	0.654	0.948	0.958
column	0.733	0.663	0.714	0.495	2,458	2,725	252	0.713	0.669	0.720	0.509	0.681	0.730
window	0.730	0.900	0.895	0.583	529	529	10	0.761	0.900	0.895	0.624	0.699	0.900
water_dispenser	0.970	1.000	0.995	0.958	115	115	136	0.984	0.902	0.958	0.874	0.976	0.905
potted_plant	0.830	0.520	0.634	0.292	852	1,119	75	0.491	0.467	0.435	0.279	0.450	0.467
waste_bin	0.791	0.750	0.828	0.597	862	1,604	78	0.936	0.942	0.982	0.923	0.935	0.949
fire_extinguisher	0.918	0.743	0.860	0.624	10,624	2,954	2,175	0.903	0.649	0.798	0.540	0.868	0.694
wet_floor_sign	0.930	1.000	0.977	0.891	1,057	1,779	43	0.957	1.000	0.984	0.899	0.948	1.000
table	0.809	0.791	0.809	0.580	2,204	2,643	259	0.769	0.517	0.579	0.371	0.750	0.541
monitor	0.988	0.691	0.883	0.558	1,347	1,044	436	0.854	0.700	0.794	0.523	0.814	0.727

Confusion matrices. v3's matrix (Fig. 3a) has empty backpack and handbag rows and columns (no validation labels), which is why 17 classes are evaluated; v4's (Fig. 3b) has exactly 17. Diagonal entries are recall at Ultralytics' confusion-matrix operating point (confidence 0.25, IoU 0.45), whereas Table 3 gives recall at the confidence that maximises the mean F1; hence v3's chair recall is 0.30 in the matrix but 0.241 in the table (fire_extinguisher 0.82 versus 0.743). Chair is v3's weakest class: 85% of its 665 validation labels come from cluttered COCO scenes, where its recall is 0.24, against 1.00 on the Roboflow chair images. The 0.43 in v3's background column means that 43% of its false detections on background were predicted as fire_extinguisher; it is not a miss rate. Recalls of 1.00 for slide, notice_board, water_dispenser and wet_floor_sign come from small validation sets (Table 3), with heavy leakage for the first three, and do not indicate perfect real-world detection.

v4. v4 ran all 100 epochs (3.1 h, best epoch 94; Table 3). Its own-split results are lower than v3's because its split is harder: no validation image has a near-duplicate in training, and the completed COCO labels add many people, tables and plants in cluttered scenes, which lowers person, table and potted_plant in Table 3. Re-scored on the same v4 validation images, with v3's 19 outputs mapped onto the 17 classes, the mean mAP@50 rises from 0.632 to 0.827 (+0.195) on all 2,922 images and from 0.608 to 0.813 (+0.205) on a fair subset of 2,247 images with no near-duplicate in either training split (mAP@50-95 from 0.522 to 0.663 and from 0.500 to 0.646), so v3's own split (0.869) strongly favoured it. The largest recall gains (Fig. 2b) are for waste_bin, monitor, stairs, person, potted_plant and elevator. The gains for person, table and chair come mainly from the completed COCO labels (v3, trained to treat unlabelled people as background, finds only 4% of people in crowded scenes); those for stairs, elevator, monitor, potted_plant and waste_bin partly reflect the new sources and so also measure generalisation. As data and class balance changed together, the gains show the combined effect of the rebuild and cannot be attributed to the balancing alone. Fire_extinguisher recall falls by 0.07, a deliberate consequence of undersampling 4,588 fire-extinguisher-only images, and water_dispenser by 0.10 on the full set, because 127 of its 136 validation images near-duplicate v3's training images (both models score 1.00 on the nine unseen ones). For wet_floor_sign (43 of 43 detected) the two-sided 95% Clopper–Pearson lower bound is 0.92, for waste_bin (74 of 78) the 95% Wilson interval is 0.88–0.98, and window and notice_board are too small for firm conclusions.

3.2. Depth model comparison and alert quality

MiDaS Small [12] (`torch.hub`) and Depth Anything V2 Small [13] (HuggingFace Transformers), both producing relative inverse depth (Fig. 4), were compared on the 13 videos. The original comparison used separate notebooks that differed in more than the model (MiDaS: per-frame percentile thresholds, median of the inner clipped box; Depth Anything V2: fixed thresholds of 0.70 and 0.40, mean of the whole unclipped box; both with the v3 detector). Both

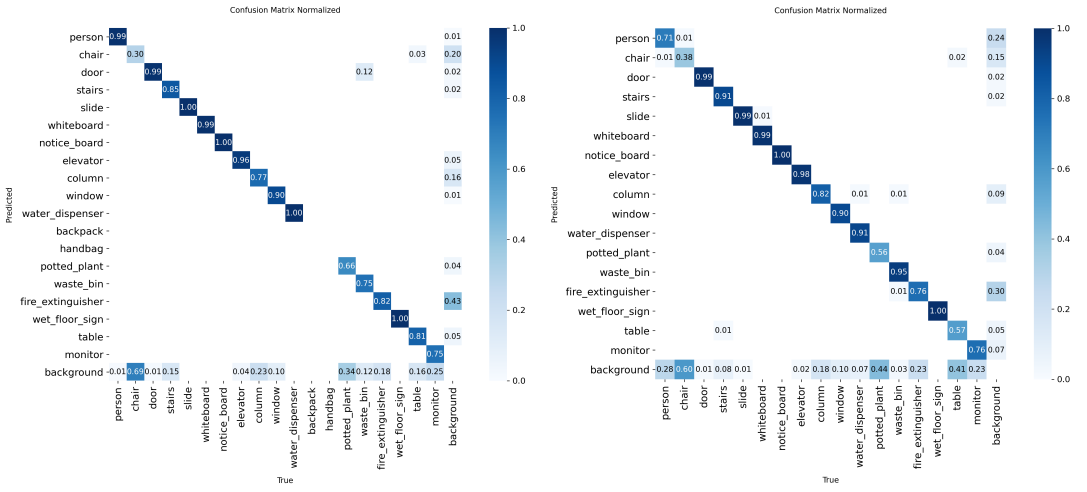


Fig. 3. Normalised confusion matrices (confidence 0.25; diagonal entries are per-class recall) of (a) Training v3, with empty backpack and handbag rows and columns, and (b) Training v4 (17 classes).

were re-run with the v4 detector in a single pass, so that frames, detections and the alert rule are identical, under both threshold rules (Table 4); the deployed configuration reproduces all 260 alerts of the pipeline exactly. Under the same rule the two models behave almost identically, whereas the fixed rule halves the alerts of Depth Anything V2 (per-video counts in Supplementary Fig. S1). The 2.3× difference reported originally (261 versus 113 alerts) therefore came from the threshold rule and box statistic, not from the depth models, and the claims that MiDaS has a higher false-positive tendency and that Depth Anything V2 gives alerts more likely to match genuine hazards were removed.

Alert quality. Every object of the 17 classes that comes within about 3 m of the camera was annotated in the 13 videos with its class, side (thirds of the frame, as in the system) and the interval during which it is that close; objects of an obstacle class (tier 1, potted plants, water dispensers) ahead count as hazards in the walking path (213 objects, 51 hazards). The annotations were drafted by an AI model from frames sampled at 2 per second, without access to any detector output, and then checked and corrected by one author. An alert is correct if an annotated object of the announced class is present within ±1 s, and a hazard is missed if no alert names its class during its interval (±1 s); proximity is not scored, as annotated distances are visual estimates. With the deployed configuration (Table 4, first row), 77% of spoken alerts are correct and 82% of these also give the correct side, but 57% of the hazards receive no spoken alert. Before the 2 s gap, 35% are missed (0.24–0.49): 11 of the 29 missed hazards are lost to the rate limit and 18 never produce an alert, of which 14 are never detected at confidence 0.40 (e.g. the water dispensers and waste bins of two waiting areas, bench seats, armchairs and two flights of stairs), 3 are always placed in the far zone and 1 was announced just before its interval. Missed hazards are therefore mainly a detection, not a depth, problem.

Path rule. Of the 64 spoken alerts, 27 concern objects ahead, 21 on the left and 16 on the right. Announcing only objects in the centre third reduces the spoken alerts to 45 and the false alerts from 3.7 to 2.7 per minute, with the same precision (0.76) and the same 29 of 51 missed in-path hazards, because the detector misses them rather than side objects crowding them out; announcing in-path objects first and side objects only when close does not help (54 alerts, 31 missed). Side alerts thus add to the alert load without improving coverage of the walking path, although they may help when the user turns (future work (3)).

3.3. End-to-end latency and frame rate

Timings were measured on the RTX 4500 Ada workstation on every second frame of the 13 videos (3,644 frames) after a 10-frame warm-up, with GPU work synchronised before each timestamp (Table 5). With MiDaS Small and both models running, the pipeline processes 13.9 frames per second. Depth estimation is the largest stage because the depth map is resized to the full 1080 × 1920 frame and copied to the CPU; fusion reaches 35 ms in cluttered frames, where two full-frame percentiles are computed per box; HEVC decoding, which a live camera avoids, adds 11 ms. For the end-to-end delay each video was streamed at its native 30 frames per second, the pipeline always processed

Table 4. Depth model comparison with the v4 detector on the 13 videos (same frames and detections in every row): the deployed percentile rule (Section 2.1) or fixed thresholds of 0.70 and 0.40, both on the inner-box median; the last row reproduces the earlier Depth Anything V2 notebook. Precision, false alerts and missed hazards (95% Wilson intervals) refer to the alerts spoken after the 2 s gap.

Depth model	Rule	Unique alerts	Spoken	Precision	False/min	Missed hazards
MiDaS Small (deployed)	percentile	260	64	0.77 (0.65–0.85)	3.7	29/51 = 0.57 (0.43–0.69)
Depth Anything V2 Small	percentile	252	64	0.80 (0.68–0.88)	3.2	27/51 = 0.53 (0.40–0.66)
MiDaS Small	fixed	257	61	0.74 (0.62–0.83)	4.0	30/51 = 0.59 (0.45–0.71)
Depth Anything V2 Small	fixed	131	38	0.71 (0.55–0.83)	2.7	38/51 = 0.75 (0.61–0.84)
Depth Anything V2 (earlier)	fixed, box mean	121	40	0.68 (0.52–0.80)	3.2	37/51 = 0.73 (0.59–0.83)

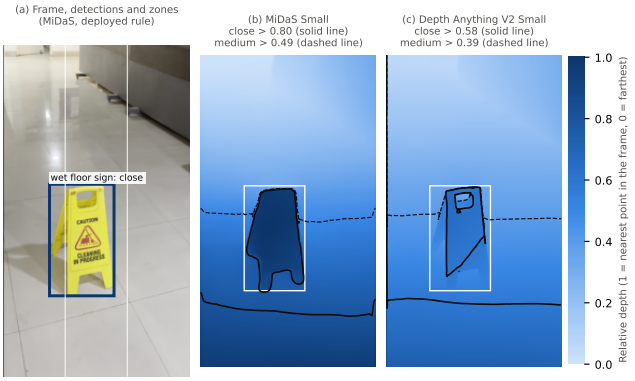


Fig. 4. Relative depth as used by EchoPath (`sign.mp4`, 2.0 s). (a) Detected box and zone. (b, c) Per-frame MiDaS Small and Depth Anything V2 Small depth (1 = nearest point of this frame) with the frame's 75th (solid, "close") and 50th (dashed, "medium") percentile contours. The wet floor sign is close because its inner-box median lies above the 75th percentile (0.98 for MiDaS); the values carry no metric scale.

the newest frame, and alerts went through the speaker of Section 2.2 with its rate limits (volume zero, which does not change the engine's timing); 75 of 228 alerts were spoken and 153 dropped by the rate limits, and the maxima were 212 ms to the alert decision and 5.6 s to speech. The sub-second delay claimed originally therefore holds only in the median: a spoken alert lasts 2.3 s while the speaker accepts a new one every 1.5 s, so alerts queue (future work (6)). The Jetson Nano has not been measured.

Depth model selection. MiDaS Small is retained. With equal alert quality under the same rule (Section 3.2), the choice rests on cost: (1) a higher and steadier frame rate, with less than half the 95th-percentile frame time (Table 5); (2) size, 86 versus 99 MB (originally given as 25 and 97 MB), or 92 versus 105 MB with YOLOv8n; (3) offline operation, which both support once their weights are cached (the original claim that Depth Anything V2 needs an internet connection was incorrect); and (4) dependencies, as MiDaS needs only PyTorch (with `timm`) and Depth Anything V2 also Transformers. The higher alert rate originally attributed to MiDaS came from the threshold rule.

4. Discussion

Evaluated without training near-duplicates, the revised model supports EchoPath's design more credibly. Among the tier-1 hazards, v4 detects most stairs and waste bins at the deployment threshold (Table 3) and all 43 wet floor signs, which, with this sample and a single source dataset, must be confirmed in the target building before it can support a safety claim. Missed stairs could cause a fall, so stairs recall is the most important figure to improve. Chair (0.38 at confidence 0.25) and table remain the weakest hazard classes, mainly in cluttered COCO scenes, followed by person in crowded scenes, and fire_extinguisher remains difficult, consistent with work on small wall-mounted objects indoors [23]; these classes need in-domain training data. For a user, about three quarters of spoken alerts name a real nearby object, but more than half of the obstacles in the walking path are not announced, partly because of the rate limit and mainly because they are not detected, and since the proximity zones are relative to each scene, "close"

Table 5. Latency on the RTX 4500 Ada GPU with detection and depth estimation running (median / 95th percentile in ms). Offline rows time each of the 3,644 processed frames; the live replay streams the videos at 30 frames per second and speaks the alerts through the deployed speaker.

Stage	MiDaS Small (deployed)	Depth Anything V2 Small
Offline, per processed frame		
Frame decoding (1080 × 1920 HEVC, CPU)	11 / 18	12 / 14
YOLOv8n detection / depth estimation	15 / 25 · 46 / 65	11 / 19 · 51 / 132
Fusion and alert decision	0.2 / 35	0.2 / 103
Pipeline per processed frame (frames per second)	71 / 109 (13.9)	72 / 237 (10.7)
Live replay at 30 frames per second, with speech (MiDaS Small; 12.7 processed frames per second)		
Frame capture to alert decision / to start of speech	85 / 137 · 616 / 3,608	–
Wait in the speech queue / length of a spoken alert	455 / 3,426 · 2,283 / 2,886	–

does not guarantee a fixed safety margin. Until missed hazards are much rarer, EchoPath should be regarded as a supplement to the white cane, not a replacement. Balancing reduces the spread among the tier-1 and tier-2 classes from more than 100:1 to about 4:1 per epoch, but notice_board (48 training labels) and the synthetic water_dispenser source (115) remain small, and no training image comes from the target building.

Future work involves (1) collecting and annotating images in the Academic Block and other buildings, and annotating the 13 videos as an in-domain test set; (2) deployment on an NVIDIA Jetson Nano with a chest-mounted camera as a wearable device; (3) a user study with visually impaired participants against a white-cane-only baseline, including whether side obstacles should be announced; (4) larger YOLOv8 variants to study the accuracy–latency trade-off on higher-compute platforms; (5) calibrating depth to metres against measured distances, so that the proximity zones correspond to distances; and (6) fewer missed hazards, through better detection of waste bins, water dispensers, unusual seating and stairs in the target building and a speech queue that neither drops newly appearing hazards nor delays them behind stale alerts.

5. Conclusion

EchoPath is an infrastructure-free, YOLOv8n-based object detection and audio navigation system for visually impaired people in indoor academic buildings. A data audit (attributed sources, no leakage, completed labels, importance-weighted balancing) raises the mean mAP@50 on identical leakage-free images from 0.608 to 0.813. The pipeline runs at 13.9 frames per second on a workstation GPU with a median delay of 0.62 s to speech, the two depth models give alerts of the same quality under identical rules, and YOLOv8n [9] is ten times smaller and 3.7 times faster than RT-DETR-L [10]. Three quarters of spoken alerts are correct, but more than half of the in-path obstacles are missed, mainly by the detector, so detection in the target building is the next priority. Students with visual impairments in Indian institutions represent a real and largely unmet need [6]; existing tools require infrastructure investment [16] or lack real-time directional guidance [18], and EchoPath's infrastructure-free design offers them an improved means of navigating campus buildings independently.

References

[1] World Health Organization. Blindness and vision impairment. WHO Fact Sheet; 2023. Available from: https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment

[2] Mannava S, Borah RR, Shamanna BR. Current estimates of the economic burden of blindness and visual impairment in India. Indian Journal of Ophthalmology; 2022. Available from: https://pmc.ncbi.nlm.nih.gov/articles/PMC9359234/

[3] Ministry of Education, Government of India. All India Survey on Higher Education 2021-22; 2022. Available from: https://cdnbbsr.s3waas.gov.in/s92049debbe566ca5782a3045cf300a3c/uploads/2025/06/2025060466438560.pdf

[4] Ministry of Education, Government of India. All India Survey on Higher Education 2019-20; 2020. Available from: https://cdnbbsr.s3waas.gov.in/s92049debbe566ca5782a3045cf300a3c/uploads/2025/06/20250604434323531.pdf

[5] An analytical report on the availability of government data for visually impaired students in India. Available from: https://www.scribd.com/document/905928671/Blind-Students-Data-India-1

[6] Bhatnagar T, Upadhyay V, Rao PVM, Marquardt N. Brilliance and resilience: a new perspective to the challenges, practices and needs of university students with visual impairments in India; 2023. Available from: https://at2030.org/challenges-practices-and-needs-of-university-students-with-visual-impairments-in-india/

[7] Abdullah, Waheed A. Opportunities and barriers to disabled students in higher education: an explorative study of visually impaired students of Aligarh Muslim University, India. Mediterranean Journal of Social Sciences; 2023. Available from: https://www.richtmann.org/journal/index.php/mjss/article/view/13225

[8] Redmon J, Divvala S, Girshick R, Farhadi A. You only look once: unified, real-time object detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition; 2016. p. 779–788.

[9] Jocher G, Chaurasia A, Qiu J. Ultralytics YOLOv8; 2023. Available from: https://github.com/ultralytics/ultralytics

[10] Zhao Y, Lv W, Xu S, Wei J, Wang G, Dang Q, et al. DETRs beat YOLOs on real-time object detection. arXiv preprint arXiv:2304.08069; 2023.

[11] Lin TY, Maire M, Belongie S, Hays J, Perona P, Ramanan D, et al. Microsoft COCO: common objects in context. In: Proceedings of the European Conference on Computer Vision; 2014. p. 740–755.

[12] Ranftl R, Lasinger K, Hafner D, Schindler K, Koltun V. Towards robust monocular depth estimation: mixing datasets for zero-shot cross-dataset transfer. IEEE Transactions on Pattern Analysis and Machine Intelligence 2022;44(3):1623–1637.

[13] Yang L, Kang B, Huang Z, Zhao Z, Xu X, Feng J, et al. Depth Anything V2. arXiv preprint arXiv:2406.09414; 2024.

[14] Bhatlawande S, Mahadevappa M, Mukherjee J, Biswas M, Das D, Gupta S. Design, development and clinical evaluation of the electronic mobility cane for vision rehabilitation. IEEE Transactions on Neural Systems and Rehabilitation Engineering; 2014.

[15] Okolo GI, Althobaiti T, Ramzan N. Smart assistive navigation system for visually impaired people. Journal of Disability Research 2025;4(1):20240086. doi:10.57197/JDR-2024-0086.

[16] Ahmetovic D, Gleason C, Ruan C, Kitani K, Takagi H, Asakawa C. NavCog: a navigational cognitive assistant for the blind. In: Proceedings of ACM MobileHCI; 2016.

[17] Ahmetovic D, Avanzini F, Baratè A, Bernareggi C, Galimberti G, Ludovico LA, et al. Sonification of rotation instructions to support navigation of people with visual impairment. In: Proceedings of IEEE PerCom; 2019.

[18] Messaoudi MD, Menelas BAJ, Mcheick H. Review of navigation assistive tools and technologies for the visually impaired. Sensors; 2022. Available from: https://pmc.ncbi.nlm.nih.gov/articles/PMC9606951/

[19] Yu X, Saniie J. Visual impairment spatial awareness system for indoor navigation and daily activities. Sensors; 2025. Available from: https://pmc.ncbi.nlm.nih.gov/articles/PMC11766877/

[20] Kedia R, Yoosuf KK, Dedeepya P, Fazal M, Arora C, Balakrishnan M. MAVI: an embedded device to assist mobility of visually impaired. In: Proceedings of the International Conference on VLSI Design; 2017.

[21] Kasture P, Tangade A, Pole A, Kumkar A, Jagtap Y. Real-time object detection using YOLO algorithm for blind people. International Journal of Advanced Research in Science, Communication and Technology 2021;5(2).

[22] Sharma T, Bali S, Yadav S. Obstacle detection system for visually impaired person. International Research Journal of Engineering and Technology 2020;7(9).

[23] Farzaneh MJ, Mohammadi HM. A blind navigation method in indoor and outdoor areas. arXiv preprint arXiv:2212.12185; 2022.

[24] Riga R. person. Roboflow Universe; 2024. Version 4. CC BY 4.0. universe.roboflow.com/rasheed-riga-ilxzv/person-fjw64

[25] Object Detect. chairs. Roboflow Universe; 2025. Version 5. CC BY 4.0. universe.roboflow.com/object-detect-zjtnj/chairs-dnhsd

[26] Nyla. Door. Roboflow Universe; 2021. Version 1. CC BY 4.0. universe.roboflow.com/nyla/door-ex6s8

[27] Dataset. Navigation. Roboflow Universe; 2025. Version 1. CC BY 4.0. universe.roboflow.com/dataset-kenli/navigation-hlzh2

[28] Student. White-board Detection. Roboflow Universe; 2024. Versions 2–4. CC BY 4.0. universe.roboflow.com/student-cbw0s/white-board-detection

[29] Library. Notice boards iiee. Roboflow Universe; 2025. Versions 1–3. CC BY 4.0. universe.roboflow.com/library-nbtst/notice-boards-iiee-ydmbd

[30] Elevator Panel detection. elevator_door. Roboflow Universe; 2026. Version 15. CC BY 4.0. universe.roboflow.com/elevator-panel-detection/elevator_door-riuxl

[31] SD. Column. Roboflow Universe; 2025. Version 2. CC BY 4.0. universe.roboflow.com/sd-ug5qc/column-2i5mg

[32] rkdf. windows. Roboflow Universe; 2024. Version 1. CC BY 4.0. universe.roboflow.com/rkdf/windows-qvnbq

[33] akaike. Water dispenser. Roboflow Universe; 2022. Version 1. CC BY 4.0. universe.roboflow.com/akaike/water-dispenser

[34] Manans Workspace. Plant Object Detection. Roboflow Universe; 2025. Version 4. CC BY 4.0. universe.roboflow.com/manans-workspace/plant-object-detection

[35] Khan U. waste-bin. Roboflow Universe; 2022. Versions 1–2. CC BY 4.0. universe.roboflow.com/umer-khan/waste-bin

[36] Workspace. Dustbin Detection. Roboflow Universe; 2025. Version 1. CC BY 4.0. universe.roboflow.com/workspace-yus7k/dustbin-detection-u76yd

[37] trialyolo. Trash_bin_detection. Roboflow Universe; 2025. Version 1. CC BY 4.0. universe.roboflow.com/trialyolo/trash_bin_detection-9glxi

[38] Kieran. Fire Extinguisher. Roboflow Universe; 2023. Version 2. CC BY 4.0. universe.roboflow.com/kieran-xejdv/fire-extinguisher-ftnny

[39] FRC 5881. wet floor. Roboflow Universe; 2024. Version 1. CC BY 4.0. universe.roboflow.com/frc-5881/wet-floor-nhjwl

[40] AIfull. table-person. Roboflow Universe; 2024. Versions 8–9. CC BY 4.0. universe.roboflow.com/aifull/table-person

[41] Object detection. Computer monitor. Roboflow Universe; 2023. Version 1. CC BY 4.0. universe.roboflow.com/object-detection-bwxix/computer-monitor

Response to Reviewers

Paper ID: 535

EchoPath: A Real-Time Assistive Indoor Navigation System For The Visually Impaired Using Computer Vision

5th International Conference on Machine Learning and Data Engineering (ICMLDE 2026)

We thank the three reviewers for their careful reading and constructive comments. Every comment is answered below. In the revised manuscript, the passages that address each comment below, and the new values in the tables, are highlighted in yellow ; values carried over from the original submission are not highlighted. Locations refer to section, table and figure numbers in the revised manuscript, which precedes this response. Several comments pointed to inconsistencies between the text, tables and figures. Following them, we audited the training data itself, and because its classes were strongly imbalanced we retrained the detector on a rebalanced dataset (Training v4); the submitted model (Training v3) is kept in the manuscript so that the effect of each change can be seen. Corrections that were not explicitly requested are listed at the end of this report. To keep the revised manuscript within the page limit, overlapping tables were merged (Tables 2 and 3), the per-video alert counts were moved to Supplementary Fig. S1 and the Discussion was combined into one section; no result was removed.

Reviewer 1

No.	Reviewer comment	Response	Location in revised manuscript
1.1	*The paper presents a relevant assistive-navigation application with a clear system overview.*	We thank the reviewer for this positive assessment.	–
1.2	*Provide dataset sizes and explain the train, validation, and test split procedure.*	Section 2.4 now reports the size and split procedure of every training run, and the new Table 2 summarises them (train / validation / test images): v1 23,445 / 2,865 / 1,367; v2 about 23,700 / 2,580 / about 1,340 (reconstructed from an archived copy of its dataset, as explained in the Table 2 caption); the submitted model (v3) 25,666 / 2,780 / none (36,765 and 5,017 labels); the revised model (v4) 20,307 / 2,922 / none (33,539 and 5,863 labels). In every version the split followed the predefined split of each source dataset, and COCO train2017 and val2017 images went to training and validation respectively. In v1 and v2 the sources' test images formed a separate test split; from v3 onwards they were merged into training. Sec. 2.4 now explains why a separate test split was not needed: no training hyperparameter was tuned on the validation split; the validation split only selects the best epoch, and the best and final epochs differ by at most 0.0014 in mAP@50 in any run, so the validation results are effectively held-out results; and the relevant test is footage from the target building, which no source contains. The 13 videos recorded there were never used for training or validation, and annotating them as a labelled test set is planned. Per-class training and validation label counts for v4 are given in Table 3 and Fig. 2a.	**Sec. 2.4; Tables 2 and 3; Fig. 2a**

No.	Reviewer comment	Response	Location in revised manuscript
1.3	*Clarify whether images from the same video or building were kept in a single split to avoid data leakage.*	We checked this directly and found that the original split did not prevent it. The public source datasets were split image by image and do not record which video a frame came from, so consecutive frames of the same video could fall into both splits. Using a perceptual difference hash (Hamming distance ≤ 4), 25.6% of the v3 validation images have a near-duplicate in the training split, concentrated in person (87% of validation labels), water_dispenser (93%), slide (56%), whiteboard (51%) and notice_board (47%). The same check gives 42.8% for v1 and at most 26.6% for v2, so the leakage affected every earlier version (Table 2). For the revised model (v4), every training image within this distance of a validation image was removed (2,727 images) while the validation split was left unchanged, and v3 and v4 are also compared on a "fair" subset of 2,247 validation images that have no near-duplicate in either model's training split (Sec. 3.1). At building level, no training or validation image comes from our test building: all come from COCO and public Roboflow datasets, and the 13 videos recorded in the building are used only for testing.	**Sec. 2.4; Table 2; Sec. 3.1**
1.4	*Reconcile reported chair-class recall: the text gives 0.30, while Table 2 reports 0.241.*	Both values are correct but were measured at different confidence thresholds, which the original manuscript did not state. The confusion matrix is computed by Ultralytics at a confidence of 0.25 (IoU 0.45), giving 0.30, whereas the per-class table (now Table 3) reports recall at the single confidence that maximises the mean F1 over all classes (0.448), giving 0.241. This is now stated in Sec. 3.1 and in the captions of Fig. 3 and Table 3; the same explanation applies to fire_extinguisher (0.82 versus 0.743). For the revised model, Table 3 now gives v4's per-class results on the same basis, and Table 3 also reports v4's precision and recall at the deployment threshold of 0.40.	**Sec. 3.1; Fig. 3 and Table 3 captions; Table 3**
1.5	*Explain why v3 was selected despite lower overall performance and substantially reduced chair detection compared with v2.*	We agree that the original justification was not supported, and it has been replaced. v2 and v3 used identical training settings and differ only in their data, and they were validated on different splits: v3's validation split (2,780 images) adds 200 COCO chair images to v2's split (2,580 images), which is also why v2 reported a chair recall of 1.000. Re-evaluating both weight files on the same v3 validation split gives an mAP@50 of 0.848 for v2 and 0.869 for v3, and 0.813 versus 0.837 after removing near-duplicate validation images; on the shared split, chair recall is 0.152 for v2 and 0.239 for v3. v3 was therefore the stronger model. Its low chair recall comes from cluttered COCO scenes (recall 0.24 on 564 COCO chair labels, against 1.00 on 101 Roboflow chair labels), and chair is now discussed as the weakest class. The revised model (v4) improves chair recall on these images (Table 3, Fig. 2b).	**Sec. 3.1; Sec. 4**
1.6	*Report end to end latency and frame rate with both object detection and depth estimation running.*	A new subsection (Sec. 3.3, Table 5) reports per-stage and end-to-end timing with detection and depth estimation running together, measured on the NVIDIA RTX 4500 Ada workstation (Ultralytics 8.4.37, PyTorch 2.4.1+cu118) over all 13 videos (3,644 processed frames, GPU synchronised before each timestamp). A frame takes 71 ms in the median (95th percentile 109 ms), i.e. 13.9 processed frames per second, of which detection takes 15 ms and depth estimation 46 ms; with Depth Anything V2 Small the pipeline runs at 10.7 frames per second. In a live replay that streams the videos at 30 frames per second and speaks the alerts, the pipeline processes 12.7 frames per second and reaches an alert decision 85 ms after frame capture (median; 95th percentile 137 ms). Speech starts 0.62 s after capture in the median and 3.6 s at the 95th percentile (response 3.3). These figures are for the workstation; the Jetson Nano has not been measured, which the text now states.	**Sec. 3.3; Table 5; Abstract; Sec. 5**

Reviewer 2

No.	Reviewer comment	Response	Location in revised manuscript
2.1	*The dataset combines COCO images with locally captured images; clarify which classes and results come from each source.*	We thank the reviewer; the original text was inaccurate on this point and has been corrected. No training or validation images were captured by the authors. All of them come from public datasets: COCO 2017, which in the revised model supplies chair, person, table, potted_plant and monitor labels in COCO scenes, and 18 public Roboflow Universe datasets, each supplying one or two classes. All 18 are now cited [24–41], and Supplementary Table S1 (attached) lists every source with its author, URL, version, licence (CC BY 4.0), classes and image counts. The 13 videos recorded in our test building are used only for testing. All detection results in Tables 2 and 3 and Figs. 2 and 3 are on the public-data validation split. The source matters most for chair: v3 reaches a recall of 0.24 on COCO chair images and 1.00 on Roboflow chair images (Sec. 3.1).	**Sec. 2.4; Refs. [24–41]; Supplementary Table S1; Sec. 3.1**
2.2	*Address the severe class imbalance, especially the very small number of waste-bin and wet-floor-sign annotations.*	We agree, and the original numbers were also wrong. The counts in the text did not match the class-distribution figure: the largest class was fire_extinguisher (10,929 training labels), not column, and the quoted 43 wet_floor_sign and 8 waste_bin annotations were validation counts. The corrected distribution is shown in the new Fig. 2a. This imbalance (10,929 fire_extinguisher against 106 waste_bin training labels) is the main reason we retrained the detector. In the revised dataset (v4) we (i) added real-world data for the scarcest important classes, raising waste_bin from 106 to 862 training labels, and replaced the stairs, elevator, potted_plant and monitor data, whose sources could not be identified, with cited sources; and (ii) applied importance-weighted class balancing (response 2.3), which reduces the spread among the hazard and landmark classes from more than 100:1 to about 4:1 per epoch. wet_floor_sign has 1,057 training labels. Table 3 now gives the training and validation label counts next to each result, and Sec. 3.1 gives confidence intervals for the smallest classes (wet_floor_sign: 43 of 43 detected, 95% lower bound 0.92; waste_bin: 74 of 78, 95% interval 0.88–0.98). The remaining limitations are stated in Sec. 4.	**Sec. 2.4; Sec. 2.5; Fig. 2a; Table 3; Sec. 3.1; Sec. 4**
2.3	*Describe the class-aware augmentation and provide evidence that it improves minority class performance.*	We must correct the original manuscript. The training logs show that no mixup, copy-paste or other class-aware augmentation was used in any run, and that v2 and v3 used identical settings. The statement has been removed, and the actual training configuration is now given in Sec. 2.5. For the revised model we implemented, and now describe, an explicit importance-weighted class-balancing scheme applied to the training split only: undersampling of over-represented, lower-importance classes, and repeat-factor sampling with $r_c = \min(3, \sqrt{T_c/f_c})$ for the hazard (tier 1) and landmark (tier 2) classes. Compared with v3 on identical leakage-free validation images, v4 improves recall for the minority classes, for example waste_bin from 0.08 to 0.95, potted_plant from 0.00 to 0.47 and elevator from 0.63 to 0.96 (Fig. 2b, Sec. 3.1). Because v4 also changes the training data (new sources, removed leakage, completed COCO labels), these gains show the combined effect of the rebuild and cannot be attributed to the balancing alone; Sec. 3.1 now states this explicitly.	**Sec. 2.5; Sec. 3.1; Table 3; Fig. 2b**

No.	Reviewer comment	Response	Location in revised manuscript
2.4	*Explain how depth estimates are calibrated to real-world distance; monocular depth values alone do not provide metric distances.*	We agree that monocular depth values alone do not give metric distances, and the manuscript now states clearly that EchoPath does not estimate them. MiDaS predicts relative inverse depth, defined only up to an unknown scale and shift. EchoPath rescales each depth map to $[0, 1]$ per frame (1 = nearest, 0 = farthest point of that frame), takes the median of the inner 50% of each detected box, and calls an object close if this value lies above the frame's 75th percentile and medium if it lies above the 50th. "Close" therefore means nearer than three quarters of the visible scene, not a distance in metres. Sec. 2.1 now describes this exactly (the original text wrongly said that the box average was used), and the new Fig. 4 shows a frame, both depth maps and the two thresholds. The consequence is stated as a limitation: the same object at the same distance can fall into different categories in different scenes. Calibration to metres against measured distances is listed as future work (Sec. 4).	**Sec. 2.1; Sec. 3.2; Fig. 4; Sec. 4**
2.5	*Use a consistent thresholding and calibration procedure when comparing MiDaS with Depth Anything V2.*	We agree. The original comparison changed several things at once: MiDaS used per-frame percentile thresholds, the median of the inner box and boxes clipped to the frame, while Depth Anything V2 used fixed thresholds (0.70/0.40) and the mean over the whole box. Both models were re-run with the v4 detector in a single pass, so that every frame, detection and alert rule is identical and only the depth model differs, and each was evaluated under both threshold rules (Table 4, Supplementary Fig. S1). The deployed configuration reproduces all 260 alerts of the pipeline exactly. Under the same rule the two models behave almost identically (260 and 252 unique alerts, 64 spoken alerts each, alert precision 0.77 and 0.80); under the fixed rule Depth Anything V2 raises 131 alerts and the earlier setup 121. The earlier 2.3$\times$ difference therefore came from the threshold rule and box statistic, not from the depth models, and the statements built on it have been removed.	**Sec. 3.2; Table 4; Supplementary Fig. S1**
2.6	*Report quantitative alert quality, such as false-alert rate and missed-hazard rate, rather than only total alert counts.*	We agree, and alert quality is now measured. Every object of the 17 classes that comes within about 3 m of the camera was annotated in the 13 videos with its class, side and time interval (213 objects); obstacles in the walking direction count as hazards (51). The annotations were drafted by an AI model from frames sampled at 2 per second, without access to any detector output, and checked and corrected by one author. An alert is correct if an annotated object of the announced class is present within ± 1 s; a hazard is missed if no alert names it during its interval. For the deployed system, 77% of spoken alerts are correct (95% interval 0.65–0.85), 82% of these give the correct side, there are 3.7 false alerts per minute, and 57% of the hazards (0.43–0.69) receive no spoken alert: 11 of the 29 missed hazards are dropped by the 2 s rate limit and 18 never produce an alert. Depth Anything V2 under the same rule scores within these intervals (0.80 precision, 3.2 false alerts per minute, 53% missed). These results are reported in Sec. 3.2 and Table 4 and discussed as a limitation in Sec. 4.	**Sec. 3.2; Table 4; Sec. 4**

Reviewer 3

No.	Reviewer comment	Response	Location in revised manuscript
3.1	*The manuscript contains conflicting performance claims, including "perfect recall" in the abstract and lower values for some classes in the results.*	We thank the reviewer. "Perfect recall" has been removed from the abstract, which now reports the revised model's mean mAP@50 and per-class recall changes on leakage-free validation images. The claims that wet_floor_sign reached a recall of 1.00 "across all three training iterations" and that four classes reached recall $\geq$0.99 "across all runs" were incorrect (in v1, wet_floor_sign recall was 0.787 and water_dispenser recall 0.297) and have been removed from the discussion and conclusion. Sec. 3.1 now explains that the values of 1.00 were measured on small validation sets with substantial train–validation leakage; Table 3 gives the number of validation labels per class; Sec. 3.1 reports confidence intervals for the smallest classes; and Sec. 4 states that these results must be confirmed in the target building.	**Abstract; Secs. 3.1, 4, 5; Table 3**
3.2	*Correct the discrepancy between the 17-class dataset description and the confusion matrix, which includes additional labels.*	The submitted model (v3) had a 19-class output head: backpack (1,238 training labels) and handbag (6) were present in the training data but had no validation labels, so only 17 classes could be evaluated, and the corresponding rows and columns of the confusion matrix are empty. Training v1 likewise had a 26-class head with 24 evaluated classes. This is now stated in the Introduction, in Sec. 2.4 (Table 2 lists the output and evaluated classes of every run) and in the caption of Fig. 3. The revised model (v4) removes these two classes, so its output head, dataset and confusion matrix all have exactly 17 classes (Fig. 3b).	**Sec. 1; Sec. 2.4; Table 2; Sec. 3.1; Fig. 3**
3.3	*The claimed sub second audio delay should be supported by measured end-to-end timing, including the speech queue and cooldown.*	The delay is now measured from frame capture to the start of speech in the live replay of response 1.6, so it includes the time an alert waits in the speech queue and the effect of the rate limits (Sec. 3.3, Table 5). The median is 0.62 s, but the 95th percentile is 3.6 s (maximum 5.6 s): a spoken alert lasts 2.3 s while the speaker accepts a new alert every 1.5 s, so alerts queue. The sub-second claim therefore holds in the median only, and Sec. 2.1 and Fig. 1 now give the measured values instead. The rate limits are now described exactly (Sec. 2.2): one alert candidate per frame (a close object before a medium one, then the most confident detection), a new alert only when its description changes, at least 1.5 s between spoken alerts, and no repetition of the same phrase within 5 s; alerts dropped by these limits produce no speech (153 of 228 in the replay). While measuring, we also found and fixed a defect in the speech module: a reused `pyttsx3` engine stopped speaking after its first phrase, so a new engine is now created for every phrase, which adds about 0.1 s.	**Secs. 2.1, 2.2, 3.3; Fig. 1; Table 5**
3.4	*Explain how the "only obstacles in the user's path" rule is implemented and validated.*	We thank the reviewer: the original sentence overstated the system. EchoPath has no filter that keeps only objects in the walking path, and the word "guarantees" has been removed. Sec. 2.1 now describes the implemented rule exactly: any close or medium object on the left, ahead or right can produce an alert (direction from the box centre at one and two thirds of the frame width), one alert candidate is taken per frame (close before medium, then the most confident detection), and its side is spoken so that the user can judge whether the object is in the way; Sec. 2.2 gives the rate limits. The rule is validated with the alert-quality evaluation of response 2.6 (Sec. 3.2): of 64 spoken alerts, 27 concern objects ahead, 21 on the left and 16 on the right. We also tested two path-aware rules on the same frames. Announcing only objects in the walking direction (centre third) reduces the spoken alerts to 45 and the false alerts from 3.7 to 2.7 per minute, with the same precision (0.76) and the same number of missed in-path hazards (29 of 51), because those hazards are missed by the detector; announcing in-path objects first and side objects only when close does not help. Whether users want side alerts is left to the planned user study (Sec. 4).	**Secs. 2.1, 2.2, 3.2, 4**

No.	Reviewer comment	Response	Location in revised manuscript
3.5	*The RT DETR comparison needs details on hardware, software versions, input size, and whether both models used the same evaluation setup.*	We agree; the original comparison ran the stock models on a single video and timed a loop that also drew and displayed every frame. It has been repeated under one setup, now fully reported in Sec. 2.3 and Table 1: NVIDIA RTX 4500 Ada, Ultralytics 8.4.37, PyTorch 2.4.1+cu118 (CUDA 11.8), 640-pixel input, batch size 1, FP32 (FP16 gave no speed-up at batch size 1), confidence 0.40, 50 warm-up predictions, the same 390 frames from the 13 videos decoded beforehand, GPU synchronised around every call, and the models timed in two alternating rounds. Both stock models (COCO-pretrained YOLOv8n and RT-DETR-L, the weights used originally) and the fine-tuned EchoPath YOLOv8n were evaluated in the same way, and accuracy was added: mAP@50 on our validation images for the five classes COCO shares with EchoPath. YOLOv8n takes 8.1 ms per frame (123 FPS) with 6.5 MB of weights and 8.7 GFLOPs; RT-DETR-L takes 29.6 ms (34 FPS) with 66.5 MB and 103.8 GFLOPs, but is more accurate before fine-tuning (mAP@50 0.410 versus 0.355). RT-DETR-L therefore fails only the size constraint, not three of four as the original text stated; this has been corrected. RT-DETR-L was not fine-tuned on our data, which the text states; the fine-tuned YOLOv8n reaches 0.575 on the same classes. The benchmark script is part of the code.	**Sec. 2.3; Table 1; Abstract; Sec. 5**
3.6	*Clarify why MiDaS is selected despite the paper reporting a higher alert rate and greater false-positive tendency.*	The higher alert rate came from the threshold rule, not from MiDaS (response 2.5): with identical rules the two models raise 260 and 252 alerts and differ by 0.03 in alert precision (0.77 versus 0.80) and 0.04 in missed-hazard rate (0.57 versus 0.53), well within the confidence intervals. The statements about a higher false-positive tendency have been removed, and the model size has been corrected (MiDaS Small 86 MB, not 25 MB; Depth Anything V2 Small 99 MB). Measured cost on the workstation: 13.9 processed frames per second with MiDaS Small against 10.7 with Depth Anything V2 Small, whose 95th-percentile frame time is twice as long (Table 5). Sec. 3.3 now gives the selection rationale on this basis: with equal alert quality, MiDaS Small is retained for its steadier frame rate (95th-percentile frame time 109 ms versus 237 ms), its slightly smaller weights and its smaller software stack (PyTorch only, against the Transformers library). The original claim that Depth Anything V2 cannot run offline was incorrect, as both models run offline once their weights are cached, and it has been corrected.	**Secs. 3.2, 3.3; Tables 4, 5**

Additional corrections

While addressing the comments above, we re-checked the manuscript against the training logs and the dataset files and made the following further corrections.

- Training configuration (Sec. 2.5): the optimiser, learning-rate schedule and augmentation now match the training logs (Ultralytics' automatic optimiser selected MuSGD; linear rather than cosine decay; no mixup or copy-paste in any run).
- Objective (3) in the Introduction referred to a hyperparameter sensitivity analysis that the paper did not contain; it now describes the data audit and retraining.
- The class-distribution figure was mislabelled and its caption described spatial statistics that the figure did not show; both have been replaced (Fig. 2a).
- Training v1 is now described correctly as a 26-class head with 24 evaluated classes, not as a 24-class model.
- Run-to-run differences are now stated as absolute changes (Table 2); the v2-to-v3 mAP@50 change is -0.039 absolute (-4.3% relative), which the original text had mixed up. Table 2 now also gives Training v1's results, so that every run is summarised.
- The per-class table (Table 3) now compares the submitted model (v3) with the revised model (v4) instead of v2 with v3; v2's overall results remain in Table 2, and v2 and v3 are compared on a shared validation split in Sec. 3.1.

- The data audit added three findings to the manuscript (Sec. 2.4): the sources of four classes could not be identified and were replaced; COCO's person, table, potted-plant and television labels were missing from the COCO chair images and have been added; and several sources contain pre-augmented copies that inflate their counts.
- All training datasets are now cited with their licence (CC BY 4.0) to meet the attribution requirement [24–41], and every source is listed in Supplementary Table S1.
- Three references in the introduction did not support their statements. The statement on YOLO accuracy for blind navigation now cites a YOLOv8 navigation study that reports it (Ref. [15]); the statement that monocular depth generalises to unseen environments now cites the MiDaS paper itself (Ref. [12]) instead of a source we could not verify; and the sentence citing an unrelated GitHub project that shares the name EchoPath has been removed.
- Sec. 2.1 described the input as 640×640 video and the box depth as the box average; it now gives the recorded resolution (1080×1920, resized to 640 pixels by YOLOv8n), the actual box statistic (median of the inner 50%) and the actual spoken phrases. The model sizes in Sec. 3.3 have been corrected from the checkpoint files (MiDaS Small 86 MB rather than 25 MB; Depth Anything V2 Small 99 MB).
- The statement that the class imbalance is inherent to the target environment has been removed (Sec. 4), and Future Work now notes that the training data contains no images from the target building (Sec. 4).